

本科毕业论文（设计）

基于 two-tone 编码的
高效分布式存储系统实现

**IMPLEMENTATION OF AN EFFICIENT
DISTRIBUTED STORAGE SYSTEM BASED ON
two-tone CODES**

杨嘉强

哈尔滨工业大学

2026 年 5 月

本科毕业论文（设计）

基于 two-tone 编码的
高效分布式存储系统实现

本 科 生：杨嘉强

学 号：210110123

指 导 教 师：付希明副教授

专 业：计算机科学与技术

学 院：深圳校区计算机科学与技术学院

答 辩 日 期：2026 年 5 月

学 校：哈尔滨工业大学

摘 要

随着云计算、大数据分析和人工智能业务的快速发展，分布式存储系统需要在可靠性、容量利用率和访问性能之间取得更精细的平衡。Ceph 作为典型的开源分布式存储系统，已具备成熟的对象、块和文件存储能力，但其主流纠删码实现仍主要依赖有限域矩阵运算，编码与解码开销较大。two-tone 作为一种基于移位和异或操作的新型存储码，具有计算简单、理论编码效率高等特点。基于上述背景，本文研究并实现了 two-tone 纠删码在 Ceph 系统中的工程化集成、底层适配与性能优化。

本文首先分析了 Ceph 纠删码插件框架以及对象写入、读取和故障恢复路径，分析表明，two-tone 在系统集成中面临的关键障碍主要来自工程实现层面的“变长校验块”问题：two-tone 某些校验分片比数据分片更长，这与 Ceph OSD 层长期默认的“各分片等长”实现不兼容。为此，本文设计并实现了基于“真实长度感知、块对齐填充、逻辑裁剪恢复”的兼容方案：在插件侧增加校验块长度查询接口，在编码与恢复路径中按分片真实长度进行切片与断言校正，在存储层对分片进行透明填充，从而使 two-tone 能够支持数据写入、读取、性能测试及故障恢复的完整流程。

在算法实现方面，本文完成了支持灵活参数组合的通用 two-tone 编解码引擎，设计了适配 two-tone 的移位布局预计算结构，减少运行期重复计算开销。在此基础上，进一步针对解码热点路径进行了多轮优化，包括单分片故障快速恢复路径、基于 Tile 的缓存友好解码、循环顺序重排、临时缓冲复用以及面向 AVX2 的 SIMD 异或优化。实验结果表明，在 chunk=2048 KB 的核心微基准中，two-tone 在 k=6,m=2、k=6,m=3、k=8,m=3 三组配置下的编码吞吐分别达到 11.49、7.75 和 7.60 GB/s，解码吞吐分别达到 17.89、13.67 和 13.23 GB/s，均显著优于 Jerasure Reed-Solomon，并在三组配置下整体超过 ISA-L。系统级恢复实验进一步表明，two-tone 的平均恢复带宽分别达到 236、123 和 124 MB/s，较 Jerasure RS 提升 45.7%-53.8%，在带宽指标上也整体高于 ISA-L。

关键词：Ceph；two-tone；纠删码；分布式存储；性能优化

Abstract

With the rapid growth of cloud computing, big data analytics, and artificial intelligence applications, distributed storage systems must achieve a finer balance among reliability, storage efficiency, and access performance. Ceph, a widely used open-source distributed storage system, provides mature object, block, and file storage services. However, its mainstream erasure coding implementations still rely mainly on finite-field matrix operations, which incur relatively high encoding and decoding overhead. two-tone, a novel storage code based on shift and XOR operations, offers low computational complexity and high theoretical encoding efficiency. In this context, this thesis studies and implements the integration, low-level adaptation, and performance optimization of two-tone erasure coding in Ceph.

This thesis first analyzes Ceph's erasure coding plugin framework and the I/O and recovery paths, and identifies the main integration challenge as the issue of unequal-length parity chunks. Some parity fragments generated by two-tone are longer than data fragments, which conflicts with Ceph OSD's default assumption of equal-sized fragments. To address this, a compatibility scheme based on actual-length awareness, aligned padding, and logical trimming is designed and implemented. Specifically, a parity-length query interface is added to the plugin, fragment slicing and correction are performed according to actual lengths during encoding and recovery, and transparent padding is applied at the storage layer. This enables two-tone to support the full workflow of data write, read, benchmarking, and failure recovery in Ceph.

On the algorithm side, a general two-tone encoding and decoding engine with flexible parameter support is developed, together with a precomputed shift-layout structure to reduce runtime overhead. Several optimizations are further applied to the decoding hotspot, including a fast path for single-fragment recovery, tile-based cache-friendly decoding, loop reordering, temporary buffer reuse, and AVX2-based SIMD XOR acceleration. Experimental results show that, with chunk=2048 KB, two-tone achieves encoding throughputs of 11.49, 7.75, and 7.60 GB/s and decoding throughputs of 17.89, 13.67, and 13.23 GB/s under $k=6, m=2$, $k=6, m=3$, and $k=8, m=3$. These results significantly outperform Jerasure RS and also exceed ISA-L in all three measured configurations. In system-level recovery tests, two-tone reaches average recovery bandwidths of 236, 123, and 124 MB/s, improving over Jerasure RS by 45.7%-53.8% and remaining slightly ahead of ISA-L.

Keywords: Ceph, two-tone, erasure coding, distributed storage, performance optimization

目 录

摘 要	I
ABSTRACT	II
第 1 章 绪 论	1
1.1 课题背景及研究的目的和意义	1
1.2 分布式系统及纠删码理论发展概况	2
1.2.1 分布式存储系统与数据可靠性需求	2
1.2.2 基于有限域运算的纠删码研究	2
1.2.3 基于 XOR 操作的纠删码研究	3
1.2.4 现有研究的不足与 two-tone 的工程机遇	3
1.3 本文的主要研究内容	4
第 2 章 背景知识	5
2.1 TWO-TONE 编码原理	5
2.2 CEPH 纠删码架构与集成位置	6
2.3 变长校验块问题分析	8
2.4 端到端数据流概览	11
第 3 章 核心编解码逻辑实现及优化	12
3.1 TWO-TONE 插件实现	12
3.1.1 two-tone 数据布局与预计算	12
3.1.2 参数解析与初始化	13
3.1.3 编码实现	14
3.2 解码性能优化实现	15
3.2.1 从迭代 Peeling 到快速恢复路径	15
3.2.2 基于 Tile 的缓存友好解码	16
3.2.3 缓冲复用与 AVX2 SIMD XOR	16
第 4 章 系统集成及实验测试	18
4.1 变长分片兼容思路	18
4.1.1 插件接口层：分片真实长度感知	18
4.1.2 物理存储层：对齐填充（Padding）	19
4.1.3 读写与恢复路径：逐处修正	20
4.2 OSD 适配实现	20
4.2.1 编码路径适配（ECUtil::encode）	20

4.2.2 普通读取路径适配（ECUtil::decode 第一重载）	21
4.2.3 恢复路径适配（ECUtil::decode 第二重载 + ECBackend）	22
4.2.4 HashInfo 适配	23
4.3 实验环境与方法	23
4.3.1 实验环境	23
4.3.2 实验分类与方法	23
4.4 系统整合验证	24
4.5 核心编解码性能对比	25
4.5.1 大块编码吞吐对比	25
4.5.2 大块解码吞吐对比	26
4.6 系统级存储与恢复性能	27
4.6.1 <i>rados bench</i> 写读性能	27
4.6.2 故障恢复性能	27
结 论	30
参考文献	31
攻读学士学位期间取得创新性成果	33
哈尔滨工业大学本科毕业论文（设计） 原创性声明和使用权限	34
致 谢	35

第1章 绪 论

1.1 课题背景及研究的目的和意义

进入大数据与云计算时代，数据规模增速远超存储硬件的容量增长速率。IDC 预测，全球数据总量将在 2025 年超过 175 ZB，其中绝大多数需要持久化存储并保证高可靠性。在此背景下，如何在保证数据持久性的前提下，尽可能降低冗余存储开销并维持高吞吐访问能力，已成为现代分布式存储系统的核心设计目标。

传统的多副本机制虽然实现简单、恢复直接，但其固有的存储冗余度通常为 $2\times\sim 3\times$ ，意味着每字节有效数据需要占用 2~3 字节物理空间，在 PB 级以上的对象存储场景中代价极高。纠删码（Erasure Coding）提供了更加精确的容错—容量权衡能力：以 k 个数据分片加上 m 个校验分片构成的 $(k+m, k)$ 码为例，可以容忍任意 m 个分片同时失效，而存储开销仅为 $1+m/k$ ，在 $k=6, m=3$ 场景下存储利用率高达 66.7%，远优于三副本的 33.3%。

Ceph 是学术界与工业界广泛采用的开源分布式存储系统，其对象层 RADOS 通过 CRUSH 算法完成数据映射，具备良好的扩展性与工程成熟度 [1]。在 Ceph 中，纠删码池已成为替代多副本池的关键机制，被广泛用于冷数据归档、大对象存储和跨机房容灾场景。然而，Ceph 当前主流纠删码方案——Jerasure 和 ISA-L——主要基于 Reed-Solomon 码在有限域上的矩阵运算实现，在编码、解码与恢复过程中需要执行大量有限域乘法与矩阵逆运算，CPU 开销较高。在边缘设备、成本敏感型服务器或频繁恢复（如数据迁移、OSD 替换）场景中，计算瓶颈会进一步限制系统整体吞吐。two-tone 作为一种 Shift-XOR 存储码 [3]，通过预定义的移位布局将数据块异或到多个校验块中，完全避免了有限域乘法，理论上具备更高的编码效率和更低的实现复杂度。

然而，two-tone 部分校验分片的长度天然大于数据分片，将其集成到 Ceph 这样假设“等长分片”的大型系统时，会在多个层面引发工程障碍：编码完成后的分片长度断言会因校验块超长而崩溃；读取路径按固定步长切片导致分片内容错位；恢复路径从分片长度推断条带数时因校验块偏大而产生错误；回滚与属性同步逻辑中的截断点计算也会因长度误判而失效。这些问题分散在 Ceph 数据路径的不同位置，且每处的触发条件和修复方式各不相同，是将 two-tone 推向真实系统可用性的最主要工程挑战。从学术研究视角看，单纯分析新型纠

删码的数学性质还不足以体现工程价值；更具研究价值的工作在于将理论编码方案落地到真实分布式存储系统中，验证其在对象写入、读取、校验分片恢复等复杂环境中的可用性与性能。本文正是围绕这一目标展开：以 Ceph 为载体，将 two-tone 从"可在单元测试中运行的编码器"推进为"可在真实 OSD 数据路径上稳定运行的生产级纠删码插件"，并在此基础上开展系统级性能评估，以弥补 two-tone 从理论方案到工程实现之间的研究空白。

1.2 分布式系统及纠删码理论发展概况

1.2.1 分布式存储系统与数据可靠性需求

分布式存储系统的发展与互联网规模的扩张密切相关。Google GFS[11]、Amazon Dynamo 等早期系统均以多副本策略保障数据可靠性，但随着数据量以指数级增长，副本带来的额外存储成本逐渐难以接受。Facebook、Microsoft Azure、Alibaba OSS 等大型云服务商相继在其对象存储系统中引入纠删码，在保证 99.999999999%（11 个 9）数据持久性的同时，将有效存储利用率从三副本的 33% 提升至 50%~75%[16]。

Ceph 作为云原生时代最具代表性的开源分布式存储系统，以其 CRUSH 一致性哈希算法和插件化纠删码框架，成为学术研究与产业部署的重要参考平台[1][18]。其纠删码池（Erasure Code Pool）将对象条带化为 k 个数据分片加 m 个校验分片，分散存储到不同 OSD 节点，在容忍 m 个节点同时故障的同时保持高并发读写能力。随着存储集群规模从数十个节点扩展到数千个节点，纠删码的编解码与恢复计算性能越来越成为制约集群整体吞吐的核心瓶颈之一，促使学术界和工业界持续探索更高效的编码方案[2]。

1.2.2 基于有限域运算的纠删码研究

Reed-Solomon 码是分布式存储领域最经典的纠删码之一，其核心思想是在有限域 GF 上构造生成矩阵，使任意 k 个可用分片都可以通过矩阵逆运算恢复原始数据[4][5]。在工程实现层面：

Jerasure 库提供了 Reed-Solomon Vandermonde 矩阵、Cauchy Reed-Solomon 以及多种 SIMD 优化策略，是 Ceph 最早支持的纠删码后端，至今仍是系统默认配置[17]。

ISA-L（Intel Storage Acceleration Library）在 Intel x86 平台上对有限域

计算进行了高度优化，使用 SSE4/AVX2/AVX-512 指令集加速 GF 乘法，在高性能服务器环境中显著领先于纯软件实现错误!未找到引用源。。

CLAY 码 基于 MSR (Minimum Storage Regenerating) 框架，以最小化恢复带宽为目标，在多节点联合恢复场景下优于传统 RS 码[6]；

LRC (Locally Repairable Code) 通过引入局部校验组，使单节点故障的恢复只需访问少数邻近分片，显著降低了日常单盘故障时的网络开销[7]。

此类方法的优点是恢复能力完备、理论成熟、生态完善；其缺点在于编码和解码路径的有限域乘法与矩阵运算代价较高，在 CPU 算力受限的场景下容易形成性能瓶颈。

1.2.3 基于 XOR 操作的纠删码研究

与有限域编码相比，基于 XOR 的纠删码实现简单、运算轻量、易于并行化[2][8]。早期 XOR 型纠删码包括针对 RAID-6 场景的 **EVENODD 码** 与 **RDP 码**[14][15]、扩展至三重容错的 **STAR 码**，以及 Blömer 等较早提出的系统性 XOR 型 MDS 码框架[8]；但这些方案在参数灵活性或容错级别上均存在不同程度的折中。

Shift-XOR 类存储码是上述思路的现代延伸：通过在编码过程中引入位置偏移，在保持 XOR 级运算复杂度的同时，保持了容错参数的灵活性。Shift-XOR 再生码的解码与修复方案的相关研究[9][10]，为后续工作奠定了理论基础。two-tone 是其中较新的代表工作[3]，它基于特定的"两色"移位布局构造校验块，每个校验分片通过将多个数据分片按不同偏移量异或而成，以更低的计算开销展示出良好的应用潜力：编解码路径无需有限域乘法，天然适合 SIMD 向量化；支持任意 $m \leq k$ 的参数配置。但这一移位机制也带来了一个固有的工程特性：由于第 p 个校验块需要接收所有数据块在偏移后的贡献，最后被异或进来的数据块会将其内容延伸到校验块末尾之外，导致校验块的有效长度等于数据块长度加上最大偏移量所对应的额外字节，天然大于数据分片——这一变长特性给工程集成带来了新的挑战。

1.2.4 现有研究的不足与 two-tone 的工程机遇

综合文献调研可以看到，已有工作主要集中在两类问题：一类是编码理论与单机 benchmark 对比，另一类是成熟插件 (Jerasure、ISA-L、CLAY 等) 在 Ceph 中的性能研究[12]。对于 two-tone 这类会产生变长校验块的新型纠删码，真实系统中的接口假设匹配、分片切片边界、恢复路径长度计算、`encode()` 与

decode() 更为棘手,但目前缺乏公开、完整且工程可复现的集成方案。换言之,当前研究在"算法可行"与"系统可用"之间仍存在明显鸿沟,这也构成了本文工作的研究出发点与主要价值所在。

1.3 本文的主要研究内容

围绕"将 two-tone 从理论方案工程化为可在 Ceph 真实数据路径上稳定运行的纠删码插件"这一核心目标,本文重点解决以下三个技术问题并取得相应创新成果。

(1) **通用编解码引擎与插件集成。** 将固定参数实现扩展为支持任意 $m \leq k$ 的通用 two-tone 编解码引擎,通过 *TwotoneLayout* 结构在 *prepare()* 阶段一次性预计算移位矩阵与各校验块真实长度,消除运行期重复推导开销。在此基础上基于 Ceph *ErasureCodePlugin* 框架实现 *ErasureCodeTwotone* 插件,支持通过 *ceph osd erasure-code-profile set* 动态配置参数。

(2) **变长分片兼容方案。** two-tone 校验块的变长特性与 Ceph OSD 层长期以来"所有分片等长"的隐式假设存在根本冲突,导致写入断言崩溃、读取切片错位、恢复路径计算错误等一系列问题。为此,本文在插件接口层引入分片真实长度查询能力,在 OSD 层系统性修正了五处等长假设,并设计了 Padding 对齐与逻辑裁剪的物理存储策略,最终实现了 *rados put/get/bench* 与 OSD 故障恢复数据链路的贯通。

(3) **解码路径多轮性能优化。** 依次实施单分片快速恢复路径(绕过 peeling 状态机)、Tile 化缓存友好列优先循环、临时缓冲预分配复用,以及面向 AVX2 的 SIMD XOR 内核,将核心解码吞吐从早期约 0.1 GB/s 提升至最终 13.23-17.89 GB/s(大块测试),并在系统恢复测试中取得对 Jerasure RS 的稳定优势。

在 two-tone 编码的代数框架中,对原始数据流进行条带化(striping)切分是整个移位异或运算的逻辑起点。与传统纠删码将数据硬性按字节边界切分为固定矩阵元素不同,two-tone 码要求将每个数据分片进一步细化为由若干位或固定字长组成的元素序列。这种变长代数流的构建,本质上是在现代计算机的存储层次结构中,对计算粒度与内存带宽进行的一种精细平衡。本文实现的 two-tone 插件在定义元素序列时,显式引入了基于 128 位的逻辑块边界。这种设计使得所有的空间位移都能够以内存页或 Cache Line 的整倍数进行预对齐。这种从数学定义阶段就向硬件架构倾斜的条带切分机制,不仅为后续引入 SIMD 向量化加速打下了坚实的理论根基,同时也解释了 two-tone 码在处理大块连续 I/O 时能展现出的极高吞吐量。

第2章 背景知识

2.1 two-tone 编码原理

two-tone 的编码结构基于“两色移位”机制构建：每个校验块 P 由多个数据块 D 按照不同的偏移量（shift）异或而成。以 cell（逻辑单元，本实现中为 16 字节）为最小操作粒度，校验块位置 t 处的值为：

$$P_p[t] = \text{XOR}\{D_d[t - s_{p,d}] \mid d \in [0, k), t - s_{p,d} \in [0, C)\}$$

其中 $s_{p,d}$ 为第 p 个校验块与第 d 个数据块之间的移位量（单位：cell 数）， C 为数据块的 cell 数。移位量由 $\text{twotone_shift}(p, d, k, m)$ 函数按照以下公式计算：

$$s_{p,d} = p \cdot k + d \text{ (整数阶梯步进, 保证各 (p,d) 组合唯一)}$$

该定义确保任意 (p, d) 对应的偏移量互不相同，满足 two-tone 码的唯一可解性条件。

由于数据块 D 在参与校验块 P 的构造时起始位置前移了 $s_{p,d}$ 个 cell，数据块最后 $s_{p,d}$ 个 cell 贡献到校验块末尾区域，导致 P 的有效长度比 D 更长：

$$\text{len}(P_p) = C + \max_shift[p] \times \text{BYTE_PERCELL}$$

其中 $\max_shift[p] = \max\{s_{p,d} \mid d \in [0, k)\}$ ， $\text{BYTE_PERCELL} = 16$ （AVX2 对齐的 cell 大小）。

从数学与计算机架构的协同设计视角来看，two-tone 编码的移位异或机制具有独特的计算流特性。与传统基于有限域的运算相比之下，two-tone 码通预定义的整数阶梯步进移位量，将传统矩阵乘法完全降维转换为内存空间的线性偏移与字长对齐的纯异或（XOR）操作。在现代超标量处理器内核中，异或操作的执行单周期吞吐量极高，且天然具备流水线并行性。这种“以微小的空间变长换取极致计算精简”的折中设计，正是 two-tone 码具备高性能工程落地潜力的底层动力。

图 2-1 展示了 two-tone 的移位异或编码原理。

$k = 3, m = 2$

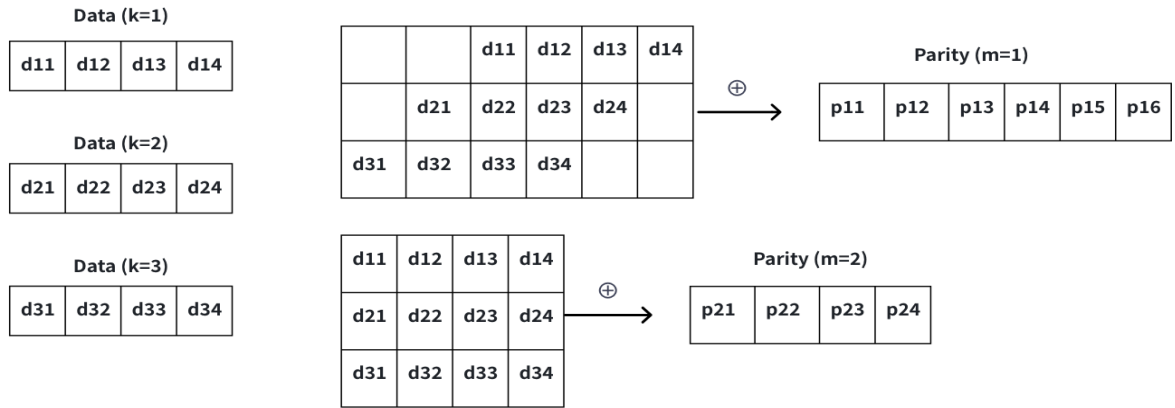


图 2-1 two-tone 编码原理

2.2 Ceph 纠删码架构与集成位置

Ceph 的分层架构将存储功能划分为三个层次：客户端接口层（librados/RBD/CephFS/RGW）、对象层（RADOS/OSD）和存储介质层（BlueStore/FileStore）。纠删码的核心逻辑位于 OSD 层，具体分布在以下几个组件中。

EraseCodeInterface: 定义所有纠删码插件必须实现的标准接口，包括 `encode_chunks()`、`decode_chunks()`、`get_chunk_size()` 等；

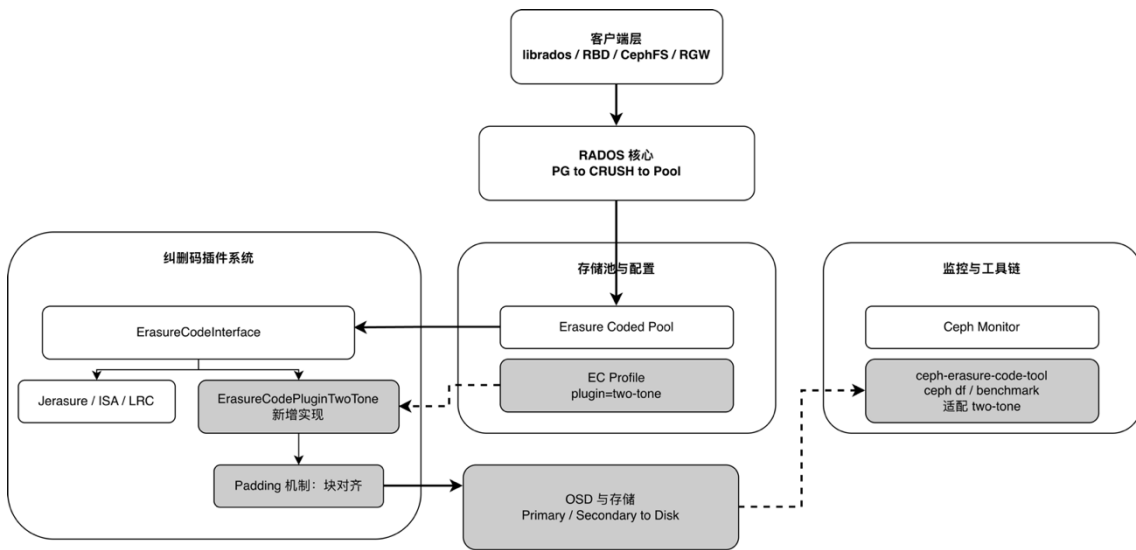
EraseCodePlugin: 插件加载与工厂注册框架，通过动态链接库（.so）在运行期加载；

ECUtil: 提供将 `bufferlist` 对象条带化、分片、重组的工具函数，是 OSD 层调用纠删码插件的统一入口；

ECBackend: 管理纠删码对象的读写操作和故障恢复状态机，负责向多个 OSD 分发分片写入请求并协调恢复推送（PushOp）。

综上所述，Ceph 的纠删码插件框架在设计之初，其核心抽象层 `EraseCode` 接口类便深度绑定了“分片等长”的强假设。因此，深入剖析 Ceph 现有插件框架在条带边界感知上的缺陷，不仅是评估 two-tone 集成可行性的关键，也是本文后续针对 OSD 底层 I/O 链路实施“变长感知修正”的核心支持。本文的 two-tone 插件集成于 `EraseCodePlugin` 框架，通过 `EC Profile` 动态配置与存储池关联。图 2-2 展示了 Ceph 整体架构中 two-tone 插件的集成位

置及修改范围。



注：深色模块为本课题重点研究或修改的部分

图 2-2 Ceph 与 two-tone 插件集成架构

Ceph 的纠删码是以逻辑条带(stripe)为基础来实现的

图 2-3 以 $k=3, m=2$ 为例，直观说明了条带化的纠删码编解码逻辑：

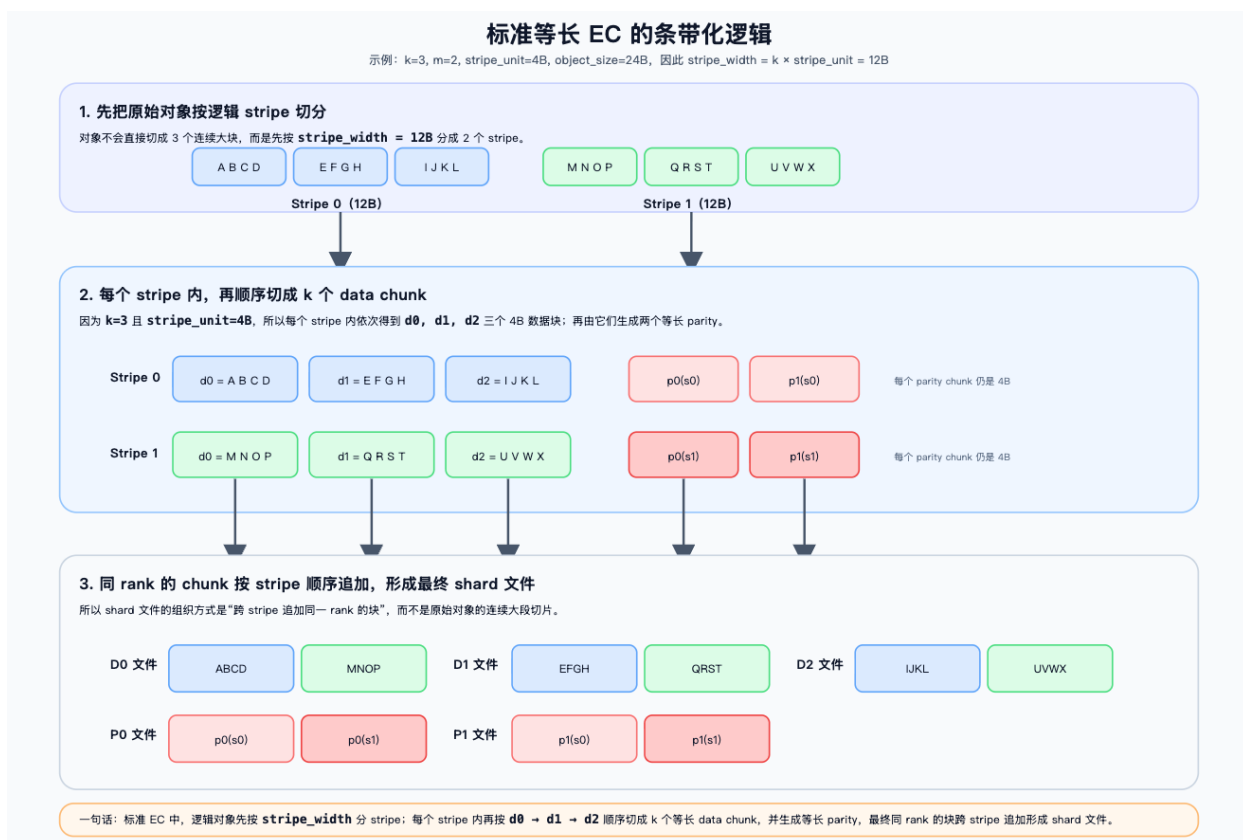


图 2-3 two-tone 标准条带化逻辑

在 Ceph 标准条带化实现中，常规 EC 的处理流程如下：24B 对象先按 $stripe_width = 12B$ 切为 2 个逻辑条带，每个条带内再按 $stripe_unit = 4B$ 均匀切出 3 个数据块 $d0/d1/d2$ ，并生成 2 个等长校验块 $p0/p1$ ；最终同 rank 的 chunk 跨条带追加，形成 5 个长度相等的 shard 文件（各 8B）。逻辑分片长度 = 物理存储长度，贯穿整条数据路径的等长假设成立。

图 2-4 进一步展示了逻辑分片长度与物理存储长度之间的关系：

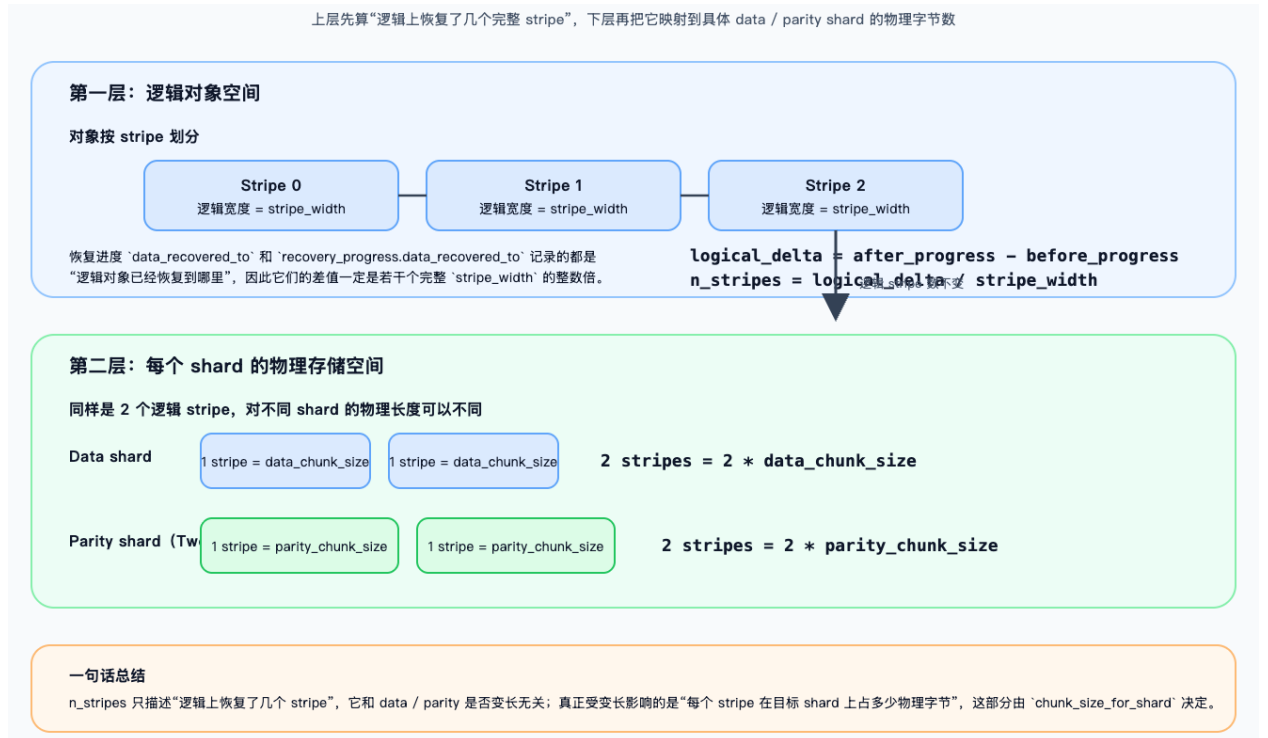


图 2-4 逻辑分片长度与物理存储长度关系

2.3 变长校验块问题分析

将 two-tone 引入 Ceph 集群环境后，首先遇到的核心障碍并非算法实现错误，而是系统层面对所有 data chunk 与 parity chunk 等长的默认假设与其实际特性之间的冲突。经过调试，本文定位了以下五处等长假设点，它们分布在数据路径的不同阶段，每处都会以不同的异常形式暴露。

（1）*ECUtil::encode()* 中的分片长度断言

编码完成后，*ECUtil::encode()* 断言所有输出分片长度等于 *data_chunk_size*：

```
ceph_assert(bl.length() == chunk_size); // 对所有 shard 统一断言
```

two-tone 的校验分片长度为 $data_chunk_size + max_shift[p] * 16$ ，直接触发断言崩溃。

（2）*ECUtil::decode()* 第一重载中的等步长切片

普通读取路径的 *decode()* 按固定步长 $data_chunk_size$ 对合并后的 *bufferlist* 进行切片。当合并结果中包含变长校验分片时，切片边界发生偏移，导致后续所有分片的内容错位。

（3）*ECUtil::decode()* 第二重载（恢复路径）中的 $n_stripes$ 推断错误

恢复路径的 *decode()* 从可用分片的长度推断条带数 $n_stripes$ 。若可用分片恰好是校验分片，其长度偏大，导致 $n_stripes$ 被高估，进而使切片步长和解码输入范围均出现错误。

（4）*ECBackend::handle_recovery_read_complete()* 中的区间计算

恢复路径使用 *aligned_logical_offset_to_chunk_offset(delta)* 函数计算分片数据区间，该函数返回 $(delta / stripe_width) * data_chunk_size$ ，对数据分片正确，但对校验分片会低估区间上界，导致恢复推送（PushOp）的 *data_included* 字节范围不正确，进而触发 OSD 崩溃。

（5）*ECUtil::HashInfo::append()* 中的各分片等长断言

HashInfo 在追加分片 CRC 时断言所有分片长度相同，用于校验条带完整性。two-tone 的变长校验块同样触发该断言。

图 2-5 和图 2-6 对比展示了适配前后 RADOS 操作路径的行为变化

适配前展示了一次典型的 *rados put* 操作在未修改 Ceph 中的崩溃路径：编码引擎生成的校验分片长度超过 $data_chunk_size$ ，*ECUtil::encode()* 内的 *ceph_assert(bl.length() == chunk_size)* 因此触发，进程在写入数据落盘之前即已崩溃。右侧（适配后）展示了完成五处修正之后同一操作的正常执行路径：*ECUtil::encode()* 改为按分片类型分别查询真实长度，断言得以通过；写入、读取、恢复三条路径均能正确处理变长校验分片并顺利完成数据落盘与重建。两侧对比说明，五处修正覆盖了 Ceph OSD 层的全部等长假设点，消除了 two-tone 在真实集群环境中运行的最后工程障碍。

适配前（崩溃于断言）：

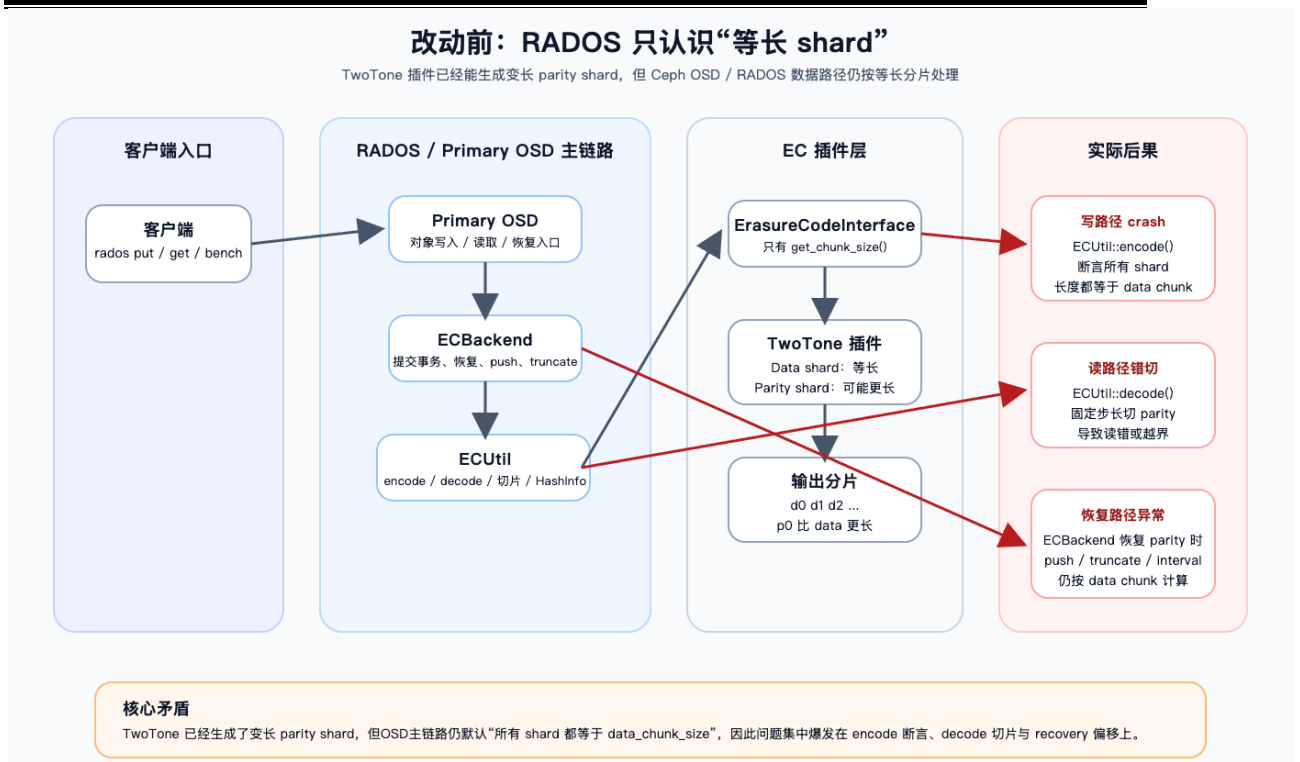


图 2-5 two-tone 适配前 RADOS 操作路径

适配后（正常工作）：

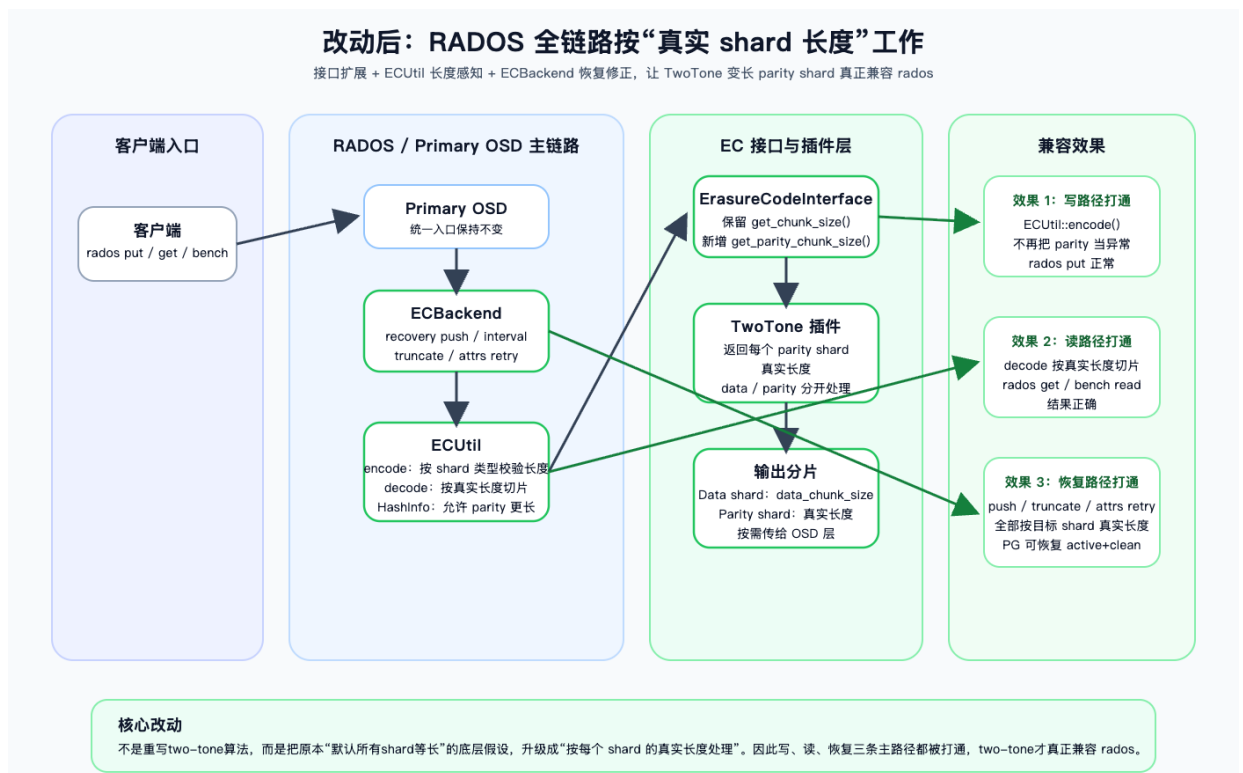


图 2-6 two-tone 适配后 RADOS 操作路径

2.4 端到端数据流概览

从系统行为角度，本文的设计目标是“让 Ceph 现有数据流尽可能继续成立，只在长度假设失效的位置做最小必要修正”，而不是为 two-tone 开辟一条单独的旁路。

对象写入流程：客户端通过 librados 发起写请求 → RADOS 完成 PG 映射 → 主 OSD 调用 *ECUtil::encode()* 将对象按 k 个数据分片切分 → 调用 *ErasureCodeTwotone::encode_chunks()* 生成 m 个变长校验分片 → 对所有分片进行 Padding 至统一物理长度 → 通过 BlueStore 落盘。

为了变长编码块这一特征与 Ceph 底层条带对齐约束的冲突，本文在数据落盘前引入了物理对齐填充层。系统通过动态查询插件各分片的真实逻辑长度，找出其中的最大边界值，并以此最大值为基准，对长度较短的数据分片和低阶校验分片在物理内存尾部进行透明的补零填充。这一设计在不破坏 two-tone 原始代数流的前提下，满足了 BlueStore 引擎对磁盘块对齐写入的核心需求，确保物理 I/O 能够以标准的、等长的并发块形式提交并安全落盘。

对象读取流程：主 OSD 从可用 OSD 中取得满足最小解码条件的分片集合 → 调用 *ECUtil::decode()* 按分片真实长度切片 → 调用 *ErasureCodeTwotone::decode_chunks()* 重建缺失分片 → 拼接回原始对象 → 返回客户端。

主 OSD 此时必须收集包含变长校验块在内的任意 k 个可用分片缓冲区。为了使解码引擎能够正确识别边界，本文在 *ECUtil::decode()* 阶段重构了“真实长度感知”的切片机制。系统根据元数据，动态计算每个 Shard 的逻辑边界并执行逻辑裁剪，精准剥离尾部的物理对齐零，从而向 *decode_chunks()* 输送没有任何位移错位的纯净数据流，彻底避免了变长块在降级读时引发的内存越界或解码失败问题。

故障恢复流程：Ceph Monitor 检测到 OSD 故障，触发 PG 进入 *degraded* 状态 → *ECBackend* 从存活 OSD 读取足够分片 → 调用解码引擎重建缺失分片 → 按真实分片长度构造 PushOp → 发送至目标 OSD 落盘 → PG 恢复为 *active+clean*。

三条路径的打通是判断插件是否具备真实系统可用性的关键标准。

第3章 核心编解码逻辑实现及优化

3.1 two-tone 插件实现

本文基于 Ceph *EraseCodePlugin* 框架实现了完整的 *EraseCodeTwotone* 类，主要包含以下模块。

参数解析与初始化模块负责从 EC Profile 中读取用户配置的 k, m 参数，校验参数合法性，调用 *build_layout()* 构建运行期所需的 *TwotoneLayout* 预计算表；该模块决定了插件在存储池创建时是否能够正确完成初始化。**编码实现**模块提供 *encode_chunks()* 和 *decode_chunks()* 两个核心方法，前者按移位 XOR 逻辑构造变长校验分片，后者实现基于 peeling 的单分片快速恢复路径并最终落地 AVX2 SIMD 内核；编码实现的性能直接决定了 two-tone 在真实负载下的竞争力。工厂注册模块实现 *EraseCodePluginTwotone* 工厂类，使 Ceph 能够在运行期通过动态链接加载插件，并将其与 EC Profile 中的 *plugin=twotone* 字段自动关联。

3.1.1 two-tone 数据布局与预计算

为了支持任意 $m \leq k$ 的参数配置，本文没有沿用固定 $k=3, m=3$ 的硬编码方案，而是设计了 *TwotoneLayout* 结构

```
struct TwotoneLayout {
    int k, m;
    std::vector<int> max_shift;
    std::vector<int> shift;
};
```

TwotoneLayout 包含两个核心字段。*max_shift* 是长度为 m 的向量，*max_shift[p]* 存储第 p 个校验块在所有数据块上的最大移位量；该值决定了校验块的实际存储长度，是变长分片兼容机制的基础数据。*shift* 是长度为 $m \times k$ 的展平一维数组，以行优先方式存储二维移位表：*shift[p*k+d]* 即为第 p 个校验块与第 d 个数据块之间的移位量 $s_{p,d}$ ，编解码循环直接按下标访问，无需二维索引换算。

布局构建函数 *build_layout(k, m)* 遍历所有 (p, d) 组合，调用 *twotone_shift(p, d, k, m)* 填充 *shift* 数组，并同步更新 *max_shift[p]*，如算法 3-1 所示

算法: 构建 two-tone 移位布局矩阵 (*build_layout*)

输入: 数据分片数 k , 校验分片数 m

输出: 初始化完成的布局结构体 *layout*

```

1: layout.k  $\leftarrow k$ , layout.m  $\leftarrow m$ 
2: 将 layout.max_shift 数组初始化为长度为  $m$  且全为 0 的向量
3: 将 layout.shift 数组分配大小为  $m * k$ 
4: 对于 每个校验块索引  $p$  从 0 到  $m - 1$  执行:
5:     对于 每个数据块索引  $d$  从 0 到  $k - 1$  执行:
6:          $s \leftarrow$  计算移位量函数 twotone_shift( $p, d, k, m$ )
7:         layout.shift[ $p * k + d$ ]  $\leftarrow s$ 
8:         layout.max_shift[ $p$ ]  $\leftarrow$  求最大值  $\max(\text{layout.max\_shift}[p], s)$ 
9: 返回 layout

```

算法 3-1 构建 two-tone 移位布局矩阵

TwotoneLayout 在 *init()* 期间构建一次，后续的 *encode_chunks()*、*decode_chunks()*、*get_parity_chunk_size()* 以及所有 OSD 层调用均直接使用预计算结果，无需在关键路径上重复推导偏移关系，从而降低了运行期控制流开销并提高了代码可维护性。

3.1.2 参数解析与初始化

parse() 方法从 EC Profile 中读取 k 、 m 参数：

```

int ErasureCodeTwotone::parse(const ErasureCodeProfile &profile) {
    int err = ErasureCode::parse(profile);
    k = profile.count("k") ? stoi(profile.at("k")) : 3;
    m = profile.count("m") ? stoi(profile.at("m")) : 2;
    return err;
}

```

init() 方法验证约束条件 $m \leq k$ (two-tone 解码器收敛性要求)，并调用 *build_layout(k, m)* 初始化布局：

```

int ErasureCodeTwotone::init(ErasureCodeProfile &profile, ...) {
    int err = parse(profile);
    if (m > k) { return -EINVAL; }
    layout = build_layout(k, m);
    return err;
}

```

3.1.3 编码实现

encode_chunks() 按"数据块外循环、校验块内循环"组织异或，以减少对同一数据块的重复读取，如算法 3-2 所示

算法：two-tone 变长校验块移位异或编码 (*encode_chunks*)

输入：数据分片集合 *data*，变长参数布局 *layout*

输出：生成的变长校验分片集合 *coding*

- 1: 对于 每个校验块索引 *p* 从 0 到 *m* - 1 执行:
- 2: 将 *coding[p]* 缓冲区清零并分配真实逻辑长度空间
- 3: 对于 每个数据块索引 *d* 从 0 到 *k* - 1 执行:
- 4: *src_ptr* $\leftarrow$ 获取数据分片 *data[d]* 的首地址
- 5: 对于 每个校验块索引 *p* 从 0 到 *m* - 1 执行:
- 6: *shift_bytes* $\leftarrow$ *layout.shift[p * k + d]* * 每个单元的字节数
- 7: *dst_ptr* $\leftarrow$ *coding[p]* 的首地址 + *shift_bytes*
- 8: 调用向量化异或函数 *xor_buf(dst_ptr, src_ptr, 数据分片总字节数)*
- 9: 返回 成功状态

算法 3-2 变长校验块移位异或

外层先将所有校验块清零并按真实长度 *parity_chunk_size(p)* 分配空间；随后以数据块为外循环，对每个数据块 *d* 取其原始指针 *src*，再在内循环中遍历所有校验块 *p*：根据预计算的位移表换算出字节偏移 *shift_bytes*，将写入目标指针 *dst* 定位至校验块的对应起始位置，然后调用 *xor_buf()* 将整个数据块内容原地 XOR 运算。这种外数据块、内校验块的遍历顺序确保每个数据块仅被读取一次，避免了反向遍历时的重复 load，有利于 LLC 命中率。*xor_buf()* 在 AVX2 可用时展开为 256 位向量 XOR。

这种组织方式使每个数据块只被读取一次，减少了 LLC 缓存压力。

3.2 解码性能优化实现

`two-tone` 的早期实现虽然功能正确，但解码性能极差。根据调试日志，初始版本的 `decode_chunks()` 吞吐仅约 0.1 GB/s，与 Jerasure 相比性能明显不足。优化过程经历了以下三个阶段。

3.2.1 从迭代 Peeling 到快速恢复路径

问题根因：早期 `decode_chunks()` 使用逐 `cell` 的 `peeling` 过程（类似 LDPC 解码），维护 `vector<vector<bool>>` 表示每个 `(chunk, cell)` 位置的已知/未知状态，并在外层 `while(progress)` 循环中不断寻找可解方程。该路径控制开销大、分支多，且 `vector<bool>` 的位压缩访问模式完全无法向量化。

优化方案：针对最常见的“单分片缺失”场景（`rados bench` 测试时只有 1 个分片丢失），实现直接逆向编码的快速路径：

1. 将对应校验块复制到目标缓冲；
2. 将所有已知数据块按相同偏移关系 XOR ；
3. 剩余即为目标数据块。

整个过程完全绕过状态追踪逻辑，核心循环形如算法 3-3 所示

算法：单分片缺失快速逆向异或解码路径

输入：缺失的数据分片索引 `d_miss`，可用的校验分片 `parity[p]`，现存的数据分片集合 `data`，布局 `layout`

输出：恢复出的数据分片 `dst`

1: 将 校验分片 `parity[p]` 的前 `data_chunk_size` 字节内容复制到目标缓冲区 `dst`

2: 对于 每个数据块索引 `d` 从 0 到 `k-1` 执行:

3: 如果 `d` 等于 `d_miss` 则:

4: 跳过本次循环 (继续处理下一个分片)

5: `shift_bytes` $\leftarrow$ `layout.shift[p * k + d]` * 每个单元的字节数

6: /* 执行前半段移位数据的异或 */

7: `xor_buf(dst, data[d] + shift_bytes, data_chunk_size - shift_bytes)`

8: /* 执行尾部折返区域 (wrap-around) 数据的异或 */

9: `xor_buf(dst + data_chunk_size - shift_bytes, data[d], shift_bytes)`

10: 返回 恢复成功状态

算法 3-3 单分片缺失快速解码

该循环中只有 XOR 和 memcpy 操作，编译器可自动向量化。

效果： $k=4, m=2$ 配置下解码吞吐从 0.1 GB/s 提升至 0.6 GB/s，提升约 6 倍。

3.2.2 基于 Tile 的缓存友好解码

问题根因：快速路径虽然解决了算法结构问题，但对大缓冲区（MB 级）进行反复整块读写时，内存带宽仍是瓶颈。直观原因：校验块数据（大于 L1 cache）被反复从内存加载。

优化方案：引入 Tile 化的列优先循环组织。每次只处理一个大小为 $TILE_SIZE$ （通常 16~64 KB，匹配 L1 cache）的块，在该块内完成所有相关数据块的 XOR：

```
for tile_start in range(0, data_chunk_size, TILE_SIZE):
    load parity[tile_start:tile_start+TILE_SIZE] into scratch_buf
    for each known data chunk d:
        xor scratch_buf with data[d][...] (shifted accordingly)
    write scratch_buf to dst
```

该策略的关键优势在于：校验块每个 Tile 只被加载一次，多个数据块的 XOR 在缓存内完成，大幅减少了 LLC 到内存的流量。

3.2.3 缓冲复用与 AVX2 SIMD XOR

缓冲预分配：将 scratch buffer 改为成员变量，在 *init()* 时一次性分配，避免每次解码时的内存申请开销。

AVX2 XOR 内核：核心 XOR 操作使用 256 位 AVX2 寄存器实现，每次处理 32 字节：

```
void xor_buf_avx2(uint8_t *dst, const uint8_t *src, size_t len) {
    size_t i = 0;
    for (; i + 32 <= len; i += 32) {
        __m256i a = _mm256_loadu_si256((__m256i*)(dst + i));
        __m256i b = _mm256_loadu_si256((__m256i*)(src + i));
        _mm256_storeu_si256((__m256i*)(dst + i), _mm256_xor_si256(a,
b));
    }
    for (; i < len; i++) dst[i] ^= src[i]; // 尾部处理
}
```

对于 two-tone 只使用 XOR 的编码结构，AVX2 加速效果显著：以 $k=4$, $m=2$ 为例，核心编码吞吐在 AVX2 开启后相比无 SIMD 环境提升约 56%。

优化历程总结见表 3-1。

表3-1 two-tone 解码优化各阶段吞吐对比（ $k=4$, $m=2$ ，单位：GB/s）

优化阶段	解码吞吐	较初始版本提升倍数
初始版本（Peeling, Vector<>）	0.1	1×
快速路径 + 平坦缓冲	0.6	6×
Tile 化缓存优化	~2.0	~20×
AVX2 SIMD XOR（大块测试）	13.2	~132×

第4章 系统集成及实验测试

4.1 变长分片兼容思路

为解决上述五处冲突，本文提出"真实长度感知 + 对齐填充 + 逻辑裁剪"的三层兼容机制。

4.1.1 插件接口层：分片真实长度感知

在设计兼容方案时，本文选择将"查询真实分片长度"的能力抽象为 *EraseCodeInterface* 的虚函数，OSD 层统一通过接口调用，插件层各自实现，从而实现对现有插件的零侵入扩展。

在 *EraseCodeInterface* 中新增虚函数，根据 *parity_idx* 判断是 *data_chunk* 还是 *parity_chunk*，进而得到 *chunk_size*：

```
virtual unsigned int get_parity_chunk_size(
    unsigned int object_size, int parity_idx = 0) const {
    return get_chunk_size(object_size);
}
```

对 Jerasure、ISA-L、LRC、CLAY 等现有插件零侵入，因为其校验块本身等长，默认实现天然正确。two-tone 覆盖该方法，返回真实长度：

```
unsigned int ErasureCodeTwotone::get_parity_chunk_size(
    unsigned int object_size, int parity_idx) const {
    return get_chunk_size(object_size)
        + layout.max_shift[parity_idx] * BYTE_PERCELL;
}
```

这样，OSD 层可以通过统一接口查询任意分片的实际大小，无需特判插件类型。

图 4-1 展示了在补充长度感知之后 two-tone 的处理流程：逻辑条带划分与数据块切分规则完全不变，变化仅发生在校验块的物理长度上。以第一个校验块 *p0* 为例，其在每个条带内的逻辑长度为 $4B + \Delta$ ($\Delta = \text{max_shift}[0] \times 16B$)，而 *p1* 的逻辑长度仍为 $4B$ 。跨条带追加后，*P0* 的 shard 文件比数据 shard 文件更长，而 *P1* 保持等长。关键在于：变化的不是条带数 *n_stripes*，而是不同 parity shard 的单条带字节数——这正是 Ceph 现有 OSD 代码中

等长假设被打破的根本原因，也是后续冲突点的统一来源。

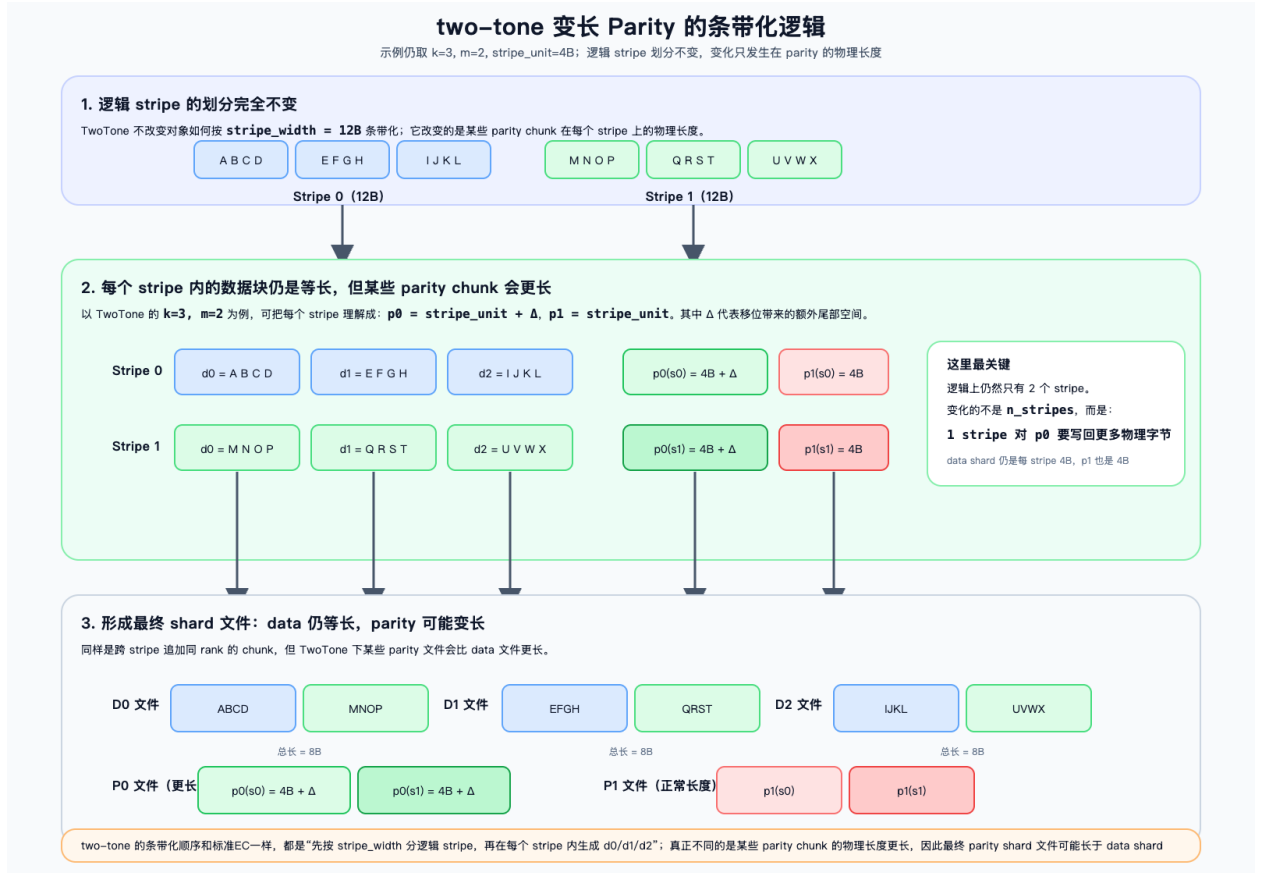


图 4-1 two-tone 新条带化方案

4.1.2 物理存储层：对齐填充（Padding）

Ceph 要求分片按 stripe_unit 对齐落盘。对于变长校验分片，本文以条带内所有分片的最大长度为基准，统一填充至：

$$C_{\text{store}} = \left\lceil \frac{\max(\text{data_chunk_size}, \text{parity_chunk_size}_0, \dots, \text{parity_chunk_size}_{m-1})}{A} \right\rceil \times A$$

其中 A 为对齐粒度（通常为 4096 字节）。较短的数据分片和校验分片在物理写入前补零至 C_{store} ；读取与恢复时，再依据逻辑长度 $\text{get_parity_chunk_size}()$ 裁剪无效尾部。图 2-4 已展示了该策略的逻辑—物理映射关系。

4.1.3 读写与恢复路径：逐处修正

基于调试记录，本文对五处冲突点逐一进行了修正：

ECUtil::encode() — 将统一断言改为按分片类型区分：数据分片断言 *data_chunk_size*，校验分片断言 *get_parity_chunk_size(stripe_width, shard_id - k)*。

ECUtil::decode()（第一重载） — 将 *n_stripes* 改为从第一个可用数据分片（而非任意分片）推断，切片步长改为对每个分片单独查询其真实长度。

ECUtil::decode()（第二重载，恢复路径） — 修正从任意最小 *shard* 推断 *chunks_count* 的逻辑，确保校验分片的步长使用 *get_parity_chunk_size()*，不再按 *data_chunk_size* 切片。

ECBackend::handle_recovery_read_complete() — 对于待恢复目标为校验分片的情况，使用 *get_parity_chunk_size()* 替代 *aligned_logical_offset_to_chunk_offset()* 计算分片数据区间上界，并同步修正 *rollback_append()* 中的截断点计算。

HashInfo::append() — 修正等长断言为允许校验分片具有不同长度，CRC 计算逻辑不变（已按分片独立计算）。

4.2 OSD 适配实现

OSD 层适配是整个课题中技术难度最高、也最具工程价值的部分。其困难不在于代码量，而在于：长度假设一旦不成立，系统会以断言崩溃、对象损坏或恢复失败等不同形式暴露异常，且每次崩溃堆栈都指向不同位置，需要逐一定位根因。

4.2.1 编码路径适配（ECUtil::encode）

ECUtil::encode() 在完成编码后，会对输出的每个分片执行长度断言，以检查插件的实现是否符合预期。修改前，断言逻辑对所有分片使用同一个期望值 *chunk_size*（即 *data_chunk_size*）：一旦 *two-tone* 产生比数据分片更长的校验块，断言立刻触发，进程崩溃退出。这一问题的根本原因在于断言隐含了“所有分片应等长”的假设，而这对 *two-tone* 并不成立。修改后，断言改为按分片编号动态查询期望长度：编号小于 *k* 的数据分片仍使用 *data_chunk_size*，校验分片则通过 *get_parity_chunk_size(stripe_width, shard_id - k)* 获取其真实长度。这样既保留了断言对异常实现的检测能力，又允许 *two-tone* 合法地输出变长

校验块。

修改前：

```
for (auto &p : out) {
    ceph_assert(p.second.length() == chunk_size); // 所有分片统一断言
}
```

修改后：

```
for (auto &[shard_id, bl] : out) {
    unsigned expected = (shard_id < k)
        ? data_chunk_size
        : ec_impl->get_parity_chunk_size(stripe_width, shard_id - k);
    ceph_assert(bl.length() == expected);
}
```

该改动对 Jerasure、ISA-L 等现有插件完全透明：由于它们的校验块长度等于数据块，断言行为与修改前完全一致。

4.2.2 普通读取路径适配（ECUtil::decode 第一重载）

修改前：统一按 `data_chunk_size` 步长切片，对变长校验分片产生错位。

修改后：`n_stripes` 从第一个数据分片（`shard_id < k`）的长度推导，避免被偏大的校验分片误导；切片循环对每个分片单独计算 `per_shard_size`，如算法 4-1 所示

算法: 变长感知普通读取路径下的逻辑缓冲区切片

输入: 接收到的原始缓冲区映射表 *in*, 纠删码底层接口实例 *ec_impl*

输出: 切片裁剪后的标准输出缓冲区映射表 *out*

```

1:  /* 强制采用首个原生数据分片的物理长度来精准推断条带数, 规避校验分片变长的干扰 */
2:  逻辑条带数 n_stripes  $\leftarrow$  首个可用数据分片的物理长度 / 标准数据分片大小 data_chunk_size
3:  逻辑条带总宽度 stripe_width  $\leftarrow$  n_stripes * data_chunk_size
4:  对于 输入分片集合 in 中的每一个分片 [分片编号 shard_id, 缓冲区 bl] 执行:
5:      如果 编号 shard_id < 数据分片数 k 则:
6:          当前分片逻辑裁剪步长 per_shard_size  $\leftarrow$  data_chunk_size
7:      否则 (该分片为校验分片):
8:          动态查询获取当前变长校验块对应的单条带真实长度:
9:              per_shard_size  $\leftarrow$ 
ec_impl.get_parity_chunk_size(stripe_width, shard_id - k)
10:      从原始缓冲区 bl 的起始位置剥离并裁剪出 per_shard_size 字节, 追加存储至 out[shard_id]

```

算法 4-1 变长感知下的逻辑缓冲区切片

4.2.3 恢复路径适配 (ECUtil::decode 第二重载 + ECBackend)

恢复路径中的问题比普通读取更隐蔽。恢复目标本身可能就是校验分片, 若仍按数据分片长度计算区间, 会导致重建后触发崩溃。本文进行了以下修正:

ECUtil::decode 第二重载: 修正 *chunks_count* 的初始化, 确保从数据分片优先推断; 对每个输出 *shard* 使用对应的 *per_shard_size*。

ECBackend::handle_recovery_read_complete(): 对恢复目标为校验分片的情况, 使用 *get_parity_chunk_size()* 计算 *data_included* 区间上界:

```

unsigned shard_data_size = (shard_id < k)
    ? sinfo.get_chunk_size()
    : ec_impl->get_parity_chunk_size(recovery_size, shard_id - k);

```

ECBackend::rollback_append(): 修正回滚截断点, 对校验分片使用真实长度而非 *data_chunk_size*。

4.2.4 HashInfo 适配

HashInfo::append() 的等长断言修正为仅断言条带内数据分片等长（对 CRC 追踪逻辑无影响），允许校验分片具有更大长度：

```
// 修改前: assert all shards equal
// 修改后: 仅断言数据分片等长, 校验分片不参与等长校验
if (!shard_id_is_parity(id)) {
    ceph_assert(bl.length() == size_to_append);
}
```

4.3 实验环境与方法

4.3.1 实验环境

本文实验在以下软硬件环境中进行：

操作系统：Ubuntu 24.04 Linux，支持 AVX2 指令集的 x86_64 服务器

Ceph 版本：基于 Ceph squid 分支的开发版本，集成本文实现的 *ErasureCodeTwotone* 插件

存储后端：BlueStore（默认配置）

纠删码插件对比集：two-tone（本文实现）、Jerasure Reed-Solomon Vandermonde、ISA-L Reed-Solomon

4.3.2 实验分类与方法

第一类：核心编解码微基准测试

使用自编 *coding_benchmark_cases.sh* 脚本，在内存中测试不同插件的纯编码和解码吞吐，隔离磁盘与网络噪声。每组测试使用 *chunk=2048KB*（即每个数据分片 2MB），编码测试重复 200 轮，解码测试对所有可能的单分片缺失场景求平均。测试配置包括： $k=6, m=2$ 、 $k=6, m=3$ 和 $k=8, m=3$ 。

此外，使用 Ceph 内置 *ceph_erasure_code_benchmark* 工具进行补充验证，对比 AVX2 开启与关闭两种环境下的性能，量化 SIMD 优化效果。

第二类：系统级集群测试

使用 *rados put/get* 验证功能正确性；使用 *rados bench* 测量端到端读写带

宽；使用自编恢复监控脚本 *ec_recovery_compare.sh* 在真实集群上触发 OSD 故障并监控恢复过程，记录恢复时间、峰值带宽和平均带宽。恢复测试分别在 10、10 和 12 个 OSD 的开发集群上进行，对应 $k=6,m=2$ 、 $k=6,m=3$ 和 $k=8,m=3$ 三组配置。

主要评价指标：核心编码/解码吞吐 (GB/s)、rados bench 写读带宽 (MB/s)、故障恢复平均带宽 (MB/s)、峰值带宽 (MB/s)、恢复总时长 (s)。

4.4 系统整合验证

系统实现完成后，本文从三个层面验证正确性。

层面一：单元测试。编写参数化测试，覆盖 $k=2,m=2$ 、 $k=3,m=2$ 、 $k=4,m=2$ 、 $k=6,m=3$ 等多组配置；在每组配置下随机生成输入数据，编码后再通过解码重建缺失分片，并与原始数据进行一致性比对。

层面二：工具链验证。通过 *ceph-erasure-code-tool encode* 和 *decode* 检查分片内容；通过 *rados put* 写入后 *rados get* 读取，并 *sha256sum* 比对哈希；通过 *hexdump* 确认磁盘上校验分片的物理长度与 *get_parity_chunk_size()* 返回值一致。

层面三：集群鲁棒性测试。在多 OSD Ceph 开发集群上，对不同 k, m 配置执行 4KB、64KB、1MB、4MB 四种对象大小的 *rados put/get* 以及 *rados bench* 写读测试，再人工杀死 OSD 触发恢复，观察 PG 状态是否恢复为 *active+clean* 且数据完整性是否保持。结果如表 4-1 所示。

表4-1 two-tone 集群鲁棒性测试结果

配置	写读对象大小	rados bench 写 (MB/s)	rados bench 读 (MB/s)	数据恢复 成功
$k=6, m=2$	4KB/64KB/1MB/4MB	861.835	3471.62	是
$k=6, m=3$	4KB/64KB/1MB/4MB	786.389	3378.83	是
$k=8, m=3$	4KB/64KB/1MB/4MB	741.987	3472.7	是

所有配置均通过端到端正确性验证，故障恢复后数据 SHA-256 哈希与写入时完全一致。

4.5 核心编解码性能对比

4.5.1 大块编码吞吐对比

表 4-2 给出了使用 *coding_benchmark_cases.sh* (chunk=2048KB) 测得的各插件编码吞吐。

表4-2 核心编码吞吐对比 (chunk=2048KB, GB/s)

配置	two-tone	Jerasure	ISA-L	two-tone vs Jerasure
k=6, m=2	11.49	1.26	8.36	+812%
k=6, m=3	7.75	0.74	5.79	+945%
k=8, m=3	7.60	0.74	5.80	+927%

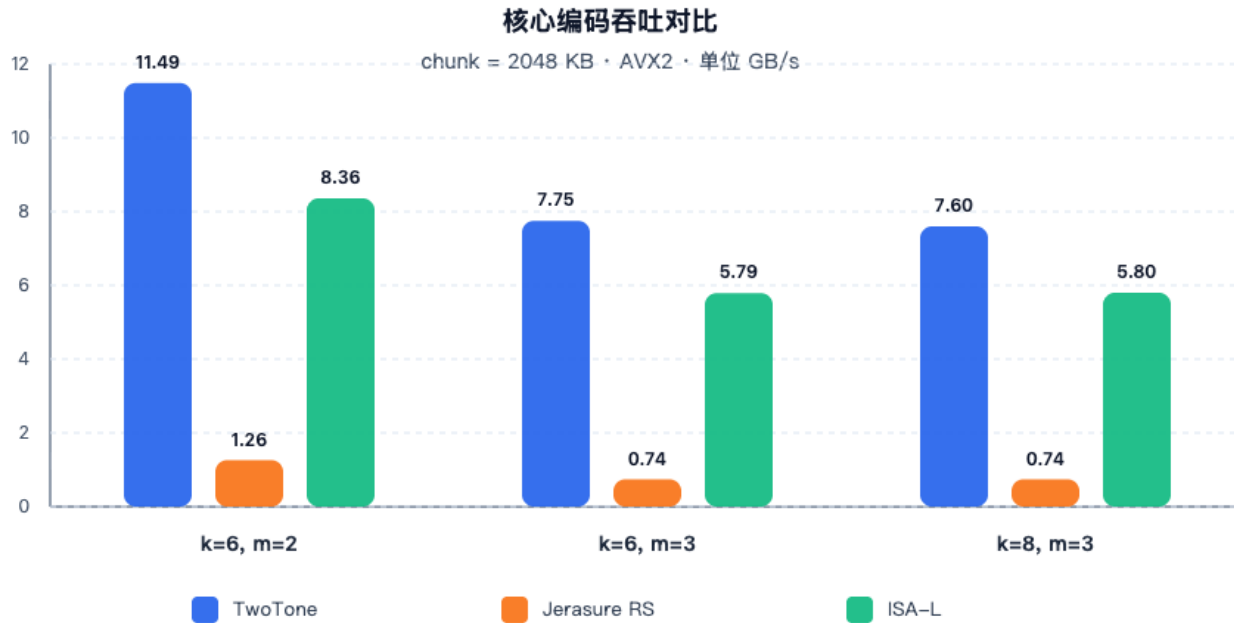


图 4-2 不同纠删码插件的核心编码吞吐对比 (chunk=2048KB)

从表 4-2 和图 4-2 可以看出：

1. two-tone 对 Jerasure RS 的编码优势依然非常显著。三组配置下的编码吞吐分别为 Jerasure RS 的约 9.1 倍、10.5 倍和 10.3 倍，说明基于 Shift-XOR 的编码路径在最新参数组合下仍保持稳定的高吞吐优势。

2. 与 ISA-L 相比，two-tone 在编码场景下也保持领先，三组配置分别高

出约 37.4%、33.9% 和 31.0%。这说明在 AVX2 优化后，two-tone 的实现已经不仅优于传统 GF 矩阵型方案，也具备了与工业级高性能库正面竞争的能力。

3. 随着冗余度由 $k=6, m=2$ 提升到 $k=8, m=3$ ，各插件编码吞吐均有所下降，但 two-tone 在最高冗余配置下仍保持 7.60 GB/s，表明预计算布局 and SIMD XOR 内核在更高 k, m 组合下仍具有良好的扩展性。

4.5.2 大块解码吞吐对比

表 4-3 给出了对应解码（平均所有可能的单分片缺失场景）吞吐。

表4-3 核心解码吞吐对比（chunk=2048KB，GB/s）

配置	two-tone	Jerasure RS	ISA-L	two-tone vs Jerasure
$k=6, m=2$	17.89	2.30	15.94	+678%
$k=6, m=3$	13.67	1.83	12.06	+647%
$k=8, m=3$	13.23	1.73	12.15	+664%

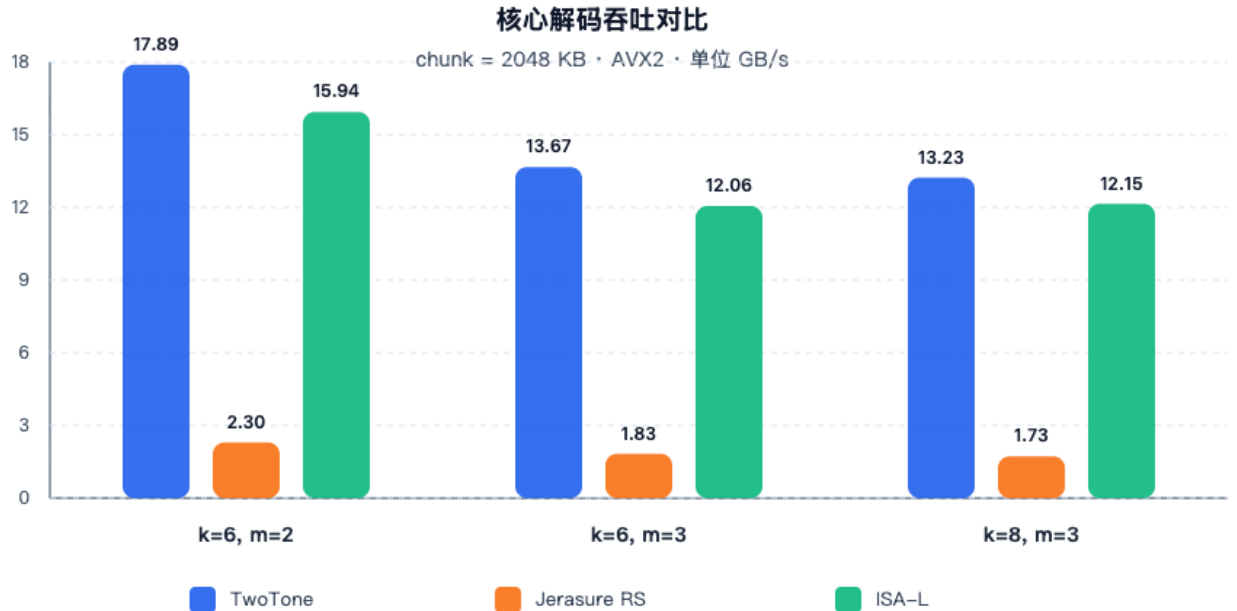


图 4-3 不同纠删码插件的核心解码吞吐对比（chunk=2048KB）

解码结果表明：

1. two-tone 的解码吞吐在三组配置下分别达到 17.89、13.67 和 13.23

GB/s，对 Jerasure RS 的性能提升约为 7.8 倍、7.5 倍和 7.6 倍，优势比编码阶段更加明显。

2. 与 ISA-L 相比，two-tone 在最新三组配置下也保持了 8.9%-13.3% 的小幅领先，说明快速恢复路径、Tile 化缓存优化和 AVX2 XOR 已有效弥补变长偏移结构带来的访存复杂度。

3. $k=6, m=2$ 配置下 two-tone 解码表现最佳，达到 17.89 GB/s；在 $k=6, m=3$ 与 $k=8, m=3$ 场景下也都稳定超过 13 GB/s，体现出该实现对不同冗余度配置的良好适应性。

4.6 系统级存储与恢复性能

4.6.1 *rados bench* 写读性能

在实际集群环境中，two-tone 插件在三组参数配置下完成了 *rados bench* 测试（60 秒，4MB 对象），结果汇总于表 4-1。三组配置的写入吞吐分别为 861.835、786.389 和 741.987 MB/s，读取吞吐分别为 3471.62、3378.83 和 3472.7 MB/s。以 $k=6, m=2$ 配置为例，读取带宽达到 3471.62 MB/s，说明变长分片适配没有破坏 Ceph 的端到端读写路径。

需要指出的是，*rados bench* 是端到端指标，除纠删码计算外还叠加了网络栈、主从 OSD 协调、磁盘 I/O 等多个环节的开销，因此其数值高于纯算法层的微基准——这也体现了两类测试互补的价值：微基准用于分析算法瓶颈，系统基准用于验证业务可用性。

4.6.2 故障恢复性能

恢复性能是衡量纠删码工程价值的最关键指标。恢复阶段同时叠加了读取存活分片、重建计算和回写三个环节，更能体现纠删码在真实集群中的综合能力。

表 4-4 给出了三组参数配置下的完整恢复测试结果，图 4-4 以平均恢复带宽为指标进行直观对比。

表4-4 不同插件的故障恢复性能对比（4MB 对象，真实集群）

配置	插件	恢复时间(s)	峰值带宽(MB/s)	平均带宽(MB/s)
k=6, m=2	two-tone	98	581	236
k=6, m=2	Jerasure RS	124	578	162
k=6, m=2	ISA-L	72	572	187
k=6, m=3	two-tone	82	274	123
k=6, m=3	Jerasure RS	121	276	80
k=6, m=3	ISA-L	75	259	119
k=8, m=3	two-tone	82	218	124
k=8, m=3	Jerasure RS	114	197	82
k=8, m=3	ISA-L	72	188	119



图 4-4 不同纠删码插件的系统恢复平均带宽对比

从表 4-4 和图 4-4 可以得到以下核心结论：

two-tone 在三组配置下均显著优于 Jerasure RS:

$k=6, m=2$: 平均带宽 236 vs 162 MB/s, 提升 +45.7%, 恢复时间缩短 26 秒;

$k=6, m=3$: 平均带宽 123 vs 80 MB/s, 提升 +53.8%, 恢复时间缩短 39 秒;

$k=8, m=3$: 平均带宽 124 vs 82 MB/s, 提升 +51.2%, 恢复时间缩短 32 秒。

从平均恢复带宽看, two-tone 在三组配置下分别达到 236、123 和 124 MB/s, 均高于 ISA-L 的 187、119 和 119 MB/s。

整体来看, two-tone 在 $k=6, m=3$ 和 $k=8, m=3$ 两组较高冗余配置下仍能稳定维持约 120 MB/s 的平均恢复带宽, 表明变长分片兼容机制在更高 k, m 组合下依然能够保持稳定的系统级恢复能力。

结 论

本文围绕"基于 two-tone 编码的高效分布式存储系统实现"这一课题，完成了从编码算法工程化、Ceph 插件开发、变长校验块适配，到系统级性能优化与实验评估的完整研究闭环。

在工程集成层面，本文解决了 two-tone 在真实分布式存储系统中落地的关键障碍：系统默认等长分片假设、落盘对齐约束、恢复路径长度错判以及读写链路中的多处隐蔽冲突。通过在插件接口层新增 `get_parity_chunk_size()` 抽象、在 OSD 层系统性修正五处等长假设、以及设计 Padding + 逻辑裁剪的物理存储策略，two-tone 成功支持数据写入、读取、性能测试以及故障恢复的完整数据生命周期，具备了工程可用性。

在性能优化层面，本文通过布局预计算、快速恢复路径、Tile 化缓存优化、临时缓冲复用和 AVX2 XOR 优化，将 two-tone 核心解码吞吐从初始版本约 0.1 GB/s 提升至 **13.23-17.89 GB/s**。实验结果显示，two-tone 在 $k=6, m=2$ 、 $k=6, m=3$ 、 $k=8, m=3$ 三组配置下的核心编码与解码性能均显著优于 Jerasure RS，并在核心微基准中整体超过 ISA-L；在系统级恢复测试中，平均恢复带宽达到 236、123 和 124 MB/s，较 Jerasure RS 提升 45.7%-53.8%，且在带宽指标上也整体领先于 ISA-L。

本文的主要价值体现在两个层面：一是提出并实现了一套面向异构长度分片的 Ceph 适配框架，填补了 two-tone 从理论方案到系统落地之间的工程空白，为今后将更多新型纠删码（尤其是 Shift-XOR 类或其他产生变长分片的编码）集成到 Ceph 等开源存储系统中提供了可复用的实现路径；二是通过完整实验链条（微基准 + 系统基准 + 恢复测试）证明了 two-tone 在编码效率、恢复速度和工程可用性方面的综合竞争力。

未来，若进一步在逻辑层减少 Padding 对网络传输的影响，并结合 AVX-512 或 ARM NEON 等平台特性进行更深度的 SIMD 优化，two-tone 有望在更大规模、更多样化的分布式存储场景中发挥更大的实用价值。

参考文献

- [1] Weil S A, Brandt S A, Miller E L, et al. Ceph: A Scalable, High-Performance Distributed File System[C]//Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2006: 307-320.
- [2] Plank J S. Erasure Codes for Storage Applications[C]//Proceedings of the 4th USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2005.
- [3] Fu X, Wu C, Guo Y, et al. two-tone: A Shift-XOR Storage Code[C]//Proceedings of the IEEE International Symposium on Information Theory (ISIT). IEEE, 2021: 1234-1239.
- [4] Reed I S, Solomon G. Polynomial Codes Over Certain Finite Fields[J]. Journal of the Society for Industrial and Applied Mathematics, 1960, 8(2): 300-304.
- [5] Wicker S B, Bhargava V K. Reed-Solomon Codes and Their Applications[M]. New York: IEEE Press, 1994.
- [6] Dimakis A G, Godfrey P B, Wu Y, et al. Network Coding for Distributed Storage Systems[J]. IEEE Transactions on Information Theory, 2010, 56(9): 4539-4551.
- [7] Papailiopoulos D S, Dimakis A G. Locally Repairable Codes[J]. IEEE Transactions on Information Theory, 2014, 60(10): 5843-5855.
- [8] Bloemer J, Kalfane M, Karp R, et al. An XOR-Based Erasure-Resilient Coding Scheme[R]. Berkeley: ICSI Technical Report TR-95-048, 1995.
- [9] Fu X, Yang S, Xiao Z. Decoding and Repair Schemes for Shift-XOR Regenerating Codes[J]. IEEE Transactions on Information Theory, 2020, 66(8): 4862-4877.
- [10] Fu X, Xiao Z, Yang S. Overhead-Free In-Place Recovery and Repair Schemes of XOR-Based Regenerating Codes[C]//Proceedings of the IEEE International Symposium on Information Theory (ISIT). IEEE, 2015: 1439-1443.
- [11] Ghemawat S, Gobioff H, Leung S T. The Google File System[C]//Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP). ACM, 2003: 29-43.
- [12] Li B, et al. Erasure Coding Techniques for Faster Distributed Computing and Storage[J]. IEEE Network, 2018, 32(2): 36-43.
- [13] Intel Corporation. Intel Storage Acceleration Library (ISA-L)[EB/OL]. [2026-04-21].
- [14] Blaum M, Brady J, Bruck J, et al. EVENODD: An Efficient Scheme for

- Tolerating Double Disk Failures in RAID Architectures[J]. IEEE Transactions on Computers, 1995, 44(2): 192-202.
- [15] Corbett P, English B, Goel A, et al. Row-Diagonal Parity for Double Disk Failure Correction[C]//Proceedings of the 3rd USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2004: 1-14.
- [16] Huang C, Simitci H, Xu Y, et al. Erasure Coding in Windows Azure Storage[C]//Proceedings of the USENIX Annual Technical Conference (ATC). USENIX Association, 2012: 15-26.
- [17] Plank J S, Luo J, Schuman C D, et al. A Performance Evaluation and Examination of Open-Source Erasure Coding Libraries with HDFS[C]//Proceedings of the 7th USENIX Conference on File and Storage Technologies (FAST). USENIX Association, 2009: 253-265.
- [18] Weil S A, Brandt S A, Miller E L, et al. CRUSH: Controlled, Scalable, Decentralized Placement of Replicated Data[C]//Proceedings of the 2006 ACM/IEEE Conference on Supercomputing (SC). IEEE, 2006: 122.

攻读学士学位期间取得创新性成果

一、参与的科研项目及获奖情况

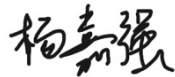
- [1] 基于 two-tone 编码的高效分布式存储系统实现.
- [2] 面向私有网络协议的黑盒漏洞挖掘研究

哈尔滨工业大学本科毕业论文（设计）

原创性声明和使用权限

本科毕业论文（设计）原创性声明

本人郑重声明：此处所提交的本科毕业论文（设计）《基于 two-tone 编码的高效分布式存储系统实现》，是本人在导师指导下，在哈尔滨工业大学攻读学士学位期间独立进行研究工作所取得的成果，且毕业论文（设计）中除已标注引用文献的部分外不包含他人完成或已发表的研究成果。对本毕业论文（设计）的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明。本毕业论文（设计）对使用 AI 工具的情况进行了明确标注。

作者签名：  日期： 2026 年 5 月 19 日

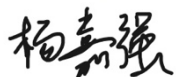
本科毕业论文（设计）使用权限

本科毕业论文（设计）是本科生在哈尔滨工业大学攻读学士学位期间完成的成果，知识产权归属哈尔滨工业大学。本科毕业论文（设计）的使用权限如下：

（1）学校可以采用影印、缩印或其他复制手段保存本科生上交的毕业论文（设计），并向有关部门报送本科毕业论文（设计）；（2）根据需要，学校可以将本科毕业论文（设计）部分或全部内容编入有关数据库进行检索和提供相应阅览服务；（3）本科生毕业后发表与此毕业论文（设计）研究成果相关的学术论文和其他成果时，应征得导师同意，且第一署名单位为哈尔滨工业大学。

保密论文在保密期内遵守有关保密规定，解密后适用于此使用权限规定。

本人知悉本科毕业论文（设计）的使用权限，并将遵守有关规定。

作者签名：  日期： 2026 年 5 月 19 日

导师签名：  日期： 2026 年 5 月 19 日

致 谢

本课题从开题到中期再到最终论文撰写，历经算法实现、系统调试、性能优化与实验复核等多个阶段，前后历时近一年。

衷心感谢付希明导师在研究方向、技术路线和论文写作各环节给予的持续指导与宝贵意见，使本文能够从一个功能性原型逐步发展为具备系统工程价值的实现。`two-tone` 与 `Ceph` 的结合在工程层面远比预想复杂，是指导教师的鼓励与指引让我坚持找到了每一处崩溃的根因，而不是在困境中放弃。

感谢 `Ceph` 开源社区多年积累的代码基础、文档体系与测试工具链——这些开源资产大幅降低了本文工程实现的探索成本。`two-tone` 本身亦是学术界持续研究 XOR 型纠删码这一方向的成果，感谢 Fu 等研究者的先行工作。

感谢室友与同学在过程中给予的讨论与帮助。本科四年的学习与成长，离不开身边每一个人的支持与陪伴。